Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

15 October 2026

gem5 TimingSimpleCPU · workload profile `test` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 8-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

Superseded results (metric schema 8). The current detector (schema 9) also counts a reload whose address lies inside a wider or merged store (e.g. two 4-byte spills read by one 8-byte load). Schema 8 missed those, so `spill_load`, `spill_event` and the `other_stack` counts here are lower (up to 15% of the spill reloads in bwaves, 8.5% in leela, 3% in x264, under 0.1% in most benchmarks); instruction counts, spill stores and the cache behaviour of the pairs it found are unaffected.

Definitions (metric schema 8). Every stack store issued inside the region of interest, and every later load that reads it back (a load from the same address at the same width), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin/m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
parest	37 262 401 281	126 952 414	16 148 158	110 058 554	745 702	12.7	0.22	0.59	9.895	67.70	1.417	1.473	0.225	0.627
deepsjeng	46 202 686 169	1 436 088 677	380 604 124	1 046 384 737	9 099 816	26.5	2.78	4.53	8.627	7.70	1.774	7.102	2.089	3.483
leela	26 063 891 644	802 403 734	20 913 175	749 189 404	32 301 155	2.6	0.34	5.75	5.161	27.19	0.646	0.422	1.011	0.368
namd	31 914 751 099	336 698 329	334 454 030	1 939 340	304 959	99.3	3.13	0.01	11.605	28.50	2.551	1.166	3.711	3.765
lbm	6 506 800 040	274 131 044	274 130 500	544	0	100.0	9.67	0.00	21.730	97.05	0.000	0.633	15.074	0.184
nab	6 691 483 051	150 911 281	124 535 928	24 348 560	2 026 793	82.5	7.90	0.73	4.074	7.87	0.156	0.129	0.628	0.732
xz	2 014 351 521	32 945 274	4 682 897	26 392 608	1 869 769	14.2	0.65	2.62	5.341	31.37	0.009	0.135	0.455	0.617
xz_4m_l1	2 001 462 938	29 215 304	3 958 947	23 548 312	1 708 045	13.6	0.57	2.35	6.788	41.37	0.010	0.167	0.994	0.605
xz_4m_l2	2 083 103 943	28 125 372	3 700 138	22 776 791	1 648 443	13.2	0.51	2.19	8.809	46.80	0.011	0.178	2.158	0.572
xz_4m_l3e	10 573 516 663	124 670 474	54 899 409	66 069 008	3 702 057	44.0	2.44	1.25	6.094	21.75	0.469	0.362	0.371	1.391
xz_4m_l4	4 011 967 585	66 558 242	20 142 751	44 411 548	2 003 943	30.3	2.11	2.21	6.791	37.27	0.233	0.751	0.552	0.305
xz_4m_l4e	12 115 711 352	127 720 342	56 348 941	67 532 844	3 838 557	44.1	2.22	1.11	5.967	19.86	0.597	0.377	0.399	1.663
xz_1m_l0	711 271 503	12 737 212	1 091 050	10 938 543	707 619	8.6	0.56	3.08	4.777	36.16	0.008	0.402	0.180	0.135
xz_1m_l1	719 848 327	12 124 762	973 253	10 458 508	693 001	8.0	0.52	2.91	5.395	42.00	0.008	0.444	0.308	0.188
xz_1m_l2	724 690 222	11 886 991	956 147	10 241 945	688 899	8.0	0.51	2.83	5.685	41.63	0.008	0.450	0.524	0.185
xz_1m_l3e	1 994 406 562	32 343 460	11 257 805	20 010 020	1 075 635	34.8	2.49	2.01	4.970	23.27	0.290	0.382	0.148	0.706
xz_1m_l4	1 076 057 785	22 448 482	5 149 009	16 486 476	812 997	22.9	1.97	3.06	5.046	32.77	0.094	0.779	0.249	0.180
xz_1m_l4e	2 198 787 666	32 786 002	11 469 277	20 222 586	1 094 139	35.0	2.33	1.84	5.016	21.50	0.391	0.391	0.154	0.842
imagick	117 734 434	2 444 393	2 450	2 437 086	4 857	0.1	0.02	4.14	4.168	86.22	7.194	58.765	0.298	0.307
blender	994 421 929	46 392 827	4 430 041	40 844 530	1 118 256	9.5	1.16	8.21	5.037	20.42	2.141	3.868	2.043	2.995
povray	2 240 986 446	168 771 227	2 121 674	155 851 654	10 797 899	1.3	0.27	13.91	8.697	47.73	6.932	19.449	1.477	1.013
omnetpp	12 426 262 471	842 378 226	5 233 032	791 025 027	46 120 167	0.6	0.09	12.73	11.570	28.59	0.053	0.720	1.230	2.191
xalancbmk	395 919 938	18 465 753	295 456	17 282 226	888 071	1.6	0.23	8.73	13.117	35.01	10.131	18.898	2.513	3.924
x264	475 535 613 470	19 188 657 460	13 318 402 468	5 322 257 634	547 997 358	69.4	6.15	2.24	1.858	32.44	0.161	0.861	0.937	1.233
gcc	16 026 870	1 020 580	52 732	952 288	15 560	5.2	0.84	11.88	5.731	38.81	0.260	1.363	0.432	0.677
mcf	24 426 982 047	234 360 896	91 410 910	142 147 076	802 910	39.0	1.58	1.16	24.643	78.48	3.545	6.915	2.675	5.246
perlbench_makerand	2 421 667 618	224 582 067	55	224 562 979	19 033	0.0	0.08	18.55	3.575	0.00	0.000	99.996	0.003	0.111
bwaves_1	17 288 358 363	145 483 579	145 451 376	26 481	5 722	100.0	3.55	0.00	26.018	37.38	5.467	0.021	3.308	4.973
bwaves_2	17 287 430 556	145 483 497	145 451 376	26 421	5 700	100.0	3.55	0.00	26.019	37.38	5.467	0.021	3.304	4.977
exchange2	57 667 143 431	1 874 700 399	1 304 759 851	345 493 349	224 447 199	69.6	5.03	1.20	1.342	2.01	1.432	2.700	3.470	2.262
fotonik3d	3 098 360 920	28 393 792	27 853 217	437 206	103 369	98.1	3.84	0.03	9.207	80.25	2.496	0.006	1.060	3.788
roms	28 955 766 090	203 000 119	182 774 988	12 238 510	7 986 621	90.0	4.79	0.08	11.833	67.05	2.452	0.885	0.302	2.456
Total	837 739 863 934	26 784 882 211	**16 553 655 165**	9 326 592 795	904 634 251	61.8	4.73	2.23	0.000	—	—	—	—	—